

Project Proposal & Methodology

Hybrid RAG & Fine-Tuning for Customer Support

GenAI Capstone · Stage 1 Deliverable

1. Introduction & Project Scope

This project designs and implements a controlled customer-support generation system built on an open-source Large Language Model (LLM), retrieval-assisted generation (RAG), and parameter-efficient fine-tuning. The work follows a structured ML/LLM lifecycle: understand the business objective, prepare instruction-tuning data and a knowledge base of corporate policies, establish a baseline, then progressively add retrieval and fine-tuning while measuring the independent impact of each.

Three progressively enhanced systems are built and compared:

- **Baseline** — the raw pre-trained model, used purely as a 'before' reference point.
- **Naive RAG (Solution V1)** — grounds answers in real company policy documents to reduce hallucination.
- **Hybrid RAG (Solution V2)** — adds a fine-tuned model that extracts a clean, structured intent from a messy customer query first, and uses that intent to drive retrieval.

2. Business Problem

Organisations increasingly rely on automated support to handle repetitive inquiries and speed up resolution. General-purpose LLMs can hold a conversation, but they frequently produce inconsistent outputs, hallucinate policy details such as refund windows, or misclassify the customer's core intent. Retrieval improves grounding by supplying relevant policy context at inference time; however, retrieval alone struggles when queries are messy, ambiguous, or emotionally charged. Combining retrieval with fine-tuning lets the system first extract the user's intent as structured data, then use that clean signal to drive the retrieval pipeline reliably.

3. Target Use Case

The headline use case for this proposal is the **ORDER** support category — the highest-volume, most repetitive class of customer contact. Within it, four representative intents are targeted:

- **place_order** — customer wants to create a new order.
- **track_order** — customer wants the status/location of an existing order.
- **change_order** — customer wants to modify an order already placed.
- **cancel_order** — customer wants to cancel an order.

The graded notebooks train and evaluate on the full 27-intent Bitext dataset so the intent router generalises; the ORDER category is the focused narrative used to define scope, boundaries, and the JSON contract below.

4. Data Description

A hybrid dataset approach supports both intent extraction and grounded generation:

- **Instruction-tuning dataset** — the *bitext/Bitext-customer-support-llm-chatbot-training-dataset*. Each record is transformed into a Prompt (raw customer query) and a target JSON solution containing intent and category.
- **Retrieval knowledge base** — 13 Markdown Standard Operating Procedure (SOP) files (e.g. refund_policy, shipping_delays, order_tracking, password_reset, technical_troubleshooting, escalation_matrix), chunked and embedded for retrieval.
- **Held-out evaluation set** — a stratified test partition created in Stage 2, guaranteeing zero data leakage. An adversarial subset is derived from the test split via regex filtering for hedging / uncertainty language.

The cleaned instruction dataset sits within the required 1,000–5,000 valid records and is partitioned into Training / Validation / Test using stratified sampling on intent, with programmatic set-intersection checks proving no duplicate prompts cross partitions.

5. Input / Output Contract

The intent router exposes a single, strict contract — analogous to a typed request/response schema in a backend API:

- **Input:** a raw customer query as a plain string (may be messy, typo-ridden, or hedging).
- **Output:** strict JSON of the form `{"intent": "...", "category": "..."} — with intent lowercase (e.g. track_order) and category uppercase (e.g. ORDER).`

The extracted intent is then normalised (underscores → spaces) and used as the query string that drives similarity search against the SOP vector store.

6. Success Criteria

Quantitative targets are fixed *before* development so evaluation is objective:

Metric	What it measures	Target
Format Adherence	Router output is syntactically valid JSON	≥ 95%
Intent Accuracy	Exact-match accuracy of extracted intent	≥ 85%
ROUGE-1 / ROUGE-L / BLEU	Semantic overlap of the final answer vs SOP-grounded reference	Improves per version
Consistency	Stability across repeated deterministic runs	≥ 90%
Hallucination Frequency	Outputs with policy violations / fabricated facts	Drops per version

7. Proposed Methodology

7.1 Baseline Model Selection

Chosen model: Qwen/Qwen2.5-1.5B-Instruct. Justification rests on three constraints. (a) **Hardware** — development runs locally on an Apple Silicon M1 Pro (MPS backend, not CUDA); a 1.5B instruct model fits comfortably in memory at float16 and supports both inference and LoRA fine-tuning on-device. (b) **Dataset characteristics** — the task is structured intent extraction plus short grounded answers, which a compact instruction-tuned model handles well; a larger model is unnecessary and would not fit the M1 budget. (c)

Fine-tuning feasibility — the model is small enough for parameter-efficient fine-tuning to converge quickly on local hardware. The same MODEL_ID is used across every notebook for a controlled comparison.

7.2 Retrieval Strategy

- **Embeddings:** sentence-transformers/all-MiniLM-L6-v2 (384-dimensional), applied to the chunked SOP corpus.
- **Vector store:** ChromaDB persisted locally at ./chroma_db/, holding the 13 embedded SOP documents.
- **Retrieval workflow:** similarity search with k=1 to select the single most relevant SOP, whose content augments the generation prompt.
- **Evaluation:** retrieval quality is judged indirectly through answer grounding (ROUGE / BLEU vs SOP-grounded references) and hallucination frequency.

7.3 Fine-Tuning Strategy

- **Method:** LoRA (Low-Rank Adaptation) via peft, with r=16, alpha=32, targeting the q_proj and v_proj attention modules.
- **Data format:** ChatML with strict JSON targets and label masking (-100) so loss is computed only on the target tokens.
- **Training:** a custom AdamW loop with validation-loss monitoring, early stopping, and best-checkpoint-by-validation-loss saving; train vs validation loss curves are plotted as evidence.
- **Deviation from spec:** LoRA is used instead of QLoRA because 4-bit bitsandbytes quantisation is CUDA-only and does not run on Apple Silicon MPS. This is a documented, hardware-driven deviation (see §9).

7.4 Evaluation Framework

All systems are evaluated under identical, deterministic inference (do_sample=False) on the same held-out test split, so differences are attributable to architecture, not chance. Five procedures are defined:

- **Format Adherence** — parse the router output; count valid JSON dicts containing an intent key.
- **Intent Accuracy** — exact match and a fuzzy (word-set) match against the gold intent.
- **ROUGE / BLEU** — overlap of the final answer against SOP-grounded references.
- **Consistency** — repeat generation and measure output stability.
- **Hallucination Frequency** — flag unsupported claims, invalid references, and policy violations across Baseline, V1, and V2.

8. Environment & Reproducibility

Component	Configuration
Compute	Apple Silicon M1 Pro — MPS backend (not CUDA)
Runtime	Local Jupyter Notebook (not Colab), Python 3.12.13, .venv
Base model	Qwen/Qwen2.5-1.5B-Instruct (float16)
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (384-dim)

Vector DB	ChromaDB at ./chroma_db/ (13 SOP files chunked + embedded)
Fine-tuning	LoRA r=16, alpha=32, target q_proj + v_proj
Reproducibility	Fixed random_state=42 for all sampling and splits

9. Assumptions & Documented Deviations

- **LoRA, not QLoRA** — 4-bit bitsandbytes is CUDA-only; the M1 uses MPS + float16.
- **Single model, two roles** — the LoRA adapter is toggled rather than merged: enabled it acts as the JSON intent router, disabled it acts as the plain-prose answer generator. This is memory-safe on the M1 and avoids loading two separate models.
- **Adversarial subset = hedging / uncertainty language** (e.g. 'not sure', 'how do i', 'unable') rather than emotion words, because the Bitext data is typo/hedge-heavy and emotion-word filtering matched zero rows.
- **Sampled evaluation** (30–50 rows) is used for runtime on the M1; every sample is reproducible via random_state=42. A full production run would evaluate the entire held-out split.

This proposal defines scope, data, the JSON contract, measurable success criteria, and the complete modelling methodology ahead of development. Results against these criteria are reported separately in the Comparative Analysis Report.